BlueRock

Learn more @ https://bluerock.io/formal-methods

BlueRock's formal methods tools

- Are built on the state-of-the-art program logics that scale from simple functions to concurrent abstractions, and even multi-program systems.
- Align with developer intuitions allowing them to be used throughout the development lifecycle from design to implementation and testing.
- Enable modularity and re-use of specifications, verifications, and automation all geared towards accelerating understanding and verification.

Systems-oriented C++

BlueRock BRiCk supports a range of modern C++ features commonly used in systems-level code including auto, new, virtual dispatch, and template instantiations.

```
template<typename T>
auto foo(const T& t)
{ return t.x; }

int main() {
  auto x = make_unique<C>();
  return foo(*x);
}
```

Accelerate the Simple

Verification patterns address idiomatic coding patterns allowing developers to rapidly specify and verify simple code enabling them to focus more time on the more mission-critical components.

```
struct two_ints {
  int x{0};
  int y{0};
};

two_ints add(const two_ints &a,
             const two_ints &b) {
  return {a.x + b.x, a.y + b.y};
}
```

```
cpp.class "two_ints"
  from main_cpp.module
  dataclass
  { copyable
  ; movable
  ; defaultable }.
```

Support the Complex

BlueRock tools support precisely modeling complex code including concurrency, collections, and heap-oriented data structures allowing developers to find difficult to test for bugs.

```
Definition semaphoreI (o : option ghost) : mpred :=
  match o with
  | None ⇒ emp
  | Some γ ⇒
    rinv semaphore.NS γ.(lg_inv)
         (Exists (c : Z) (** counter value *)
                 (w : N) (** # of pending downs *)
                 (r : N) (** # of pending ups *)
                 (sv : N) (** value of NOVA semaphore
                 qu,
           _field "Sys::Semaphore::_count" |-> atomicR ti64 c 1 **
           pureR
             sm.value γ.(sm_sm) sv ** sm.queue γ.(sm_sm) qu **
             releaseF γ sv ** waiterF γ (N.of_nat (length qu)) **
             waiterA γ w ** releaseA γ r **
             can_upF γ w **
             boundA γ c **
             qftickets.auth γ.(lg_tickets) (r + sv) + w) **
             valueA γ (Z.to_N (Z.of_N w + c))%N ** [| (w + Z.min 0 c = r)%Z |]))
  end.
```

(Relocatable) Iris concurrency invariant

C++ state.

OS State

Ghost state for bookkeeping

Encapsulate Complexity

Develop library- and program-level specifications that encapsulate implementation complexity and directly align with developer's intuitions.

```
cpp.spec "forward_list::forward_list(const forward_list&)" with
  (\this this
   \arg{other} "other" (Vref other)
   \prepost{q ls} other |-> forward_list.R q ls
   \post          this |-> forward_list.R 1$m ls).

cpp.spec "forward_list::forward_list(forward_list&&)" with
  (\this this
   \arg{other} "other" (Vref other)
   \pre{ls} other |-> forward_list.R 1$m ls
   \post this |-> forward_list.R 1$m ls **
        other |-> forward_list.R 1$m []).

cpp.spec "forward_list::clear()" with
  (\this this
   \pre{ls} this |-> forward_list.R 1$m ls
   \post    this |-> forward_list.R 1$m []).
```

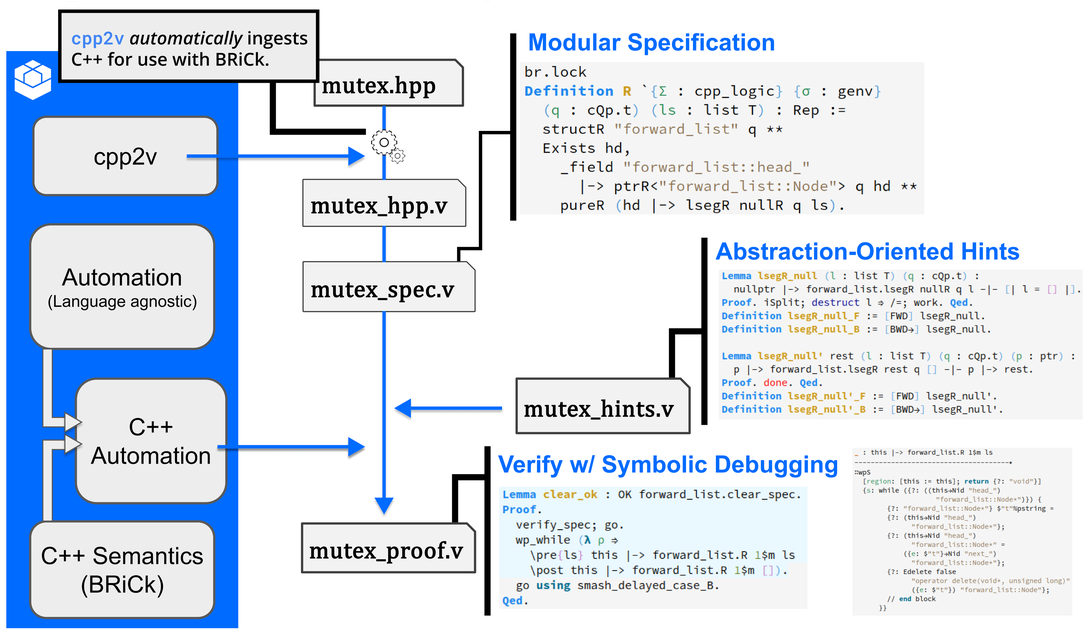
Share, Use, and Collaborate

Modular verification enables easily sharing specifications for common code with partners and collaborators. Easily import specifications written by others and share your own specifications to accelerate your users.

```
(** BlueRock's proof prelude *)
Require Import bluerock.auto.cpp.proof.
From bluerock.cpp.stdlib (** BlueRock's stdlib specs *)
Require Import cassert.spec cctype.spec.
Require Import my_code.main_cpp.
```

Verification Architecture

Modular verification enables easily sharing common code verifications with partners and collaborators. Easily import specifications written by others and share your own specifications to accelerate your users.



Why You Care. Strong Properties about Systems.

Memory Safety / Absence of UB

Establish memory safety and the absence of undefined behavior (UB) including data-race freedom. Memory safety specifications are often automatically inferable

Behavioral Refinement

Extend input / output specifications to capture the **evolution of the system over time** including its interaction with other applications.

Stronger Guarantees
(also more difficult to prove)

Application Specific

BlueRock tools support a **range of safety properties** from

- shallow, application-agnostic properties such as memory safety and the absence of undefined behavior,
- up through strong behavioral refinements.

Compose proofs of different strengths & gradually transition to stronger properties to support the mission.

Functional Correctness

EProve stronger properties about the input / output behavior of a library or function. Establish interesting class invariants.

Verifying Virtualization

Technical Reports Available

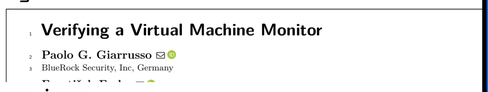
BlueRock Security has used our FM tools to verify key aspects of BlueRock's Virtualization Platform. The platform provides:

- **Virtualization-based isolation** aligning to Zero-trust principles of least-privilege.
- Mechanisms to **protect client virtual machines** from attacks.
  - Provide Kernel Integrity
  - Provide container isolation
  - Protect application runtimes, e.g. python & java

Virtual machine monitor supports running un-modified virtual machines including Linux, FreeRTOS, and VxWorks. Supports **passthrough and virtualized devices**, including VIRTIO devices.

Current verification includes:

- Page fault handler for software-emulated "memory' accesses including virtual bus logic.
- Concurrent "roundup" logic used by virtual machine introspection layer.



Communications services.

- **Uart console multiplexor (UMX)** – verified console multiplexor application providing console-level interface to applications and guests.
- **Virtual Switch (vSwitch)†** – provides VirtIO-based communication between different endpoints. Supports virtual networks. Verified routing logic for copying packets between different applications.

Verified Device Drivers.

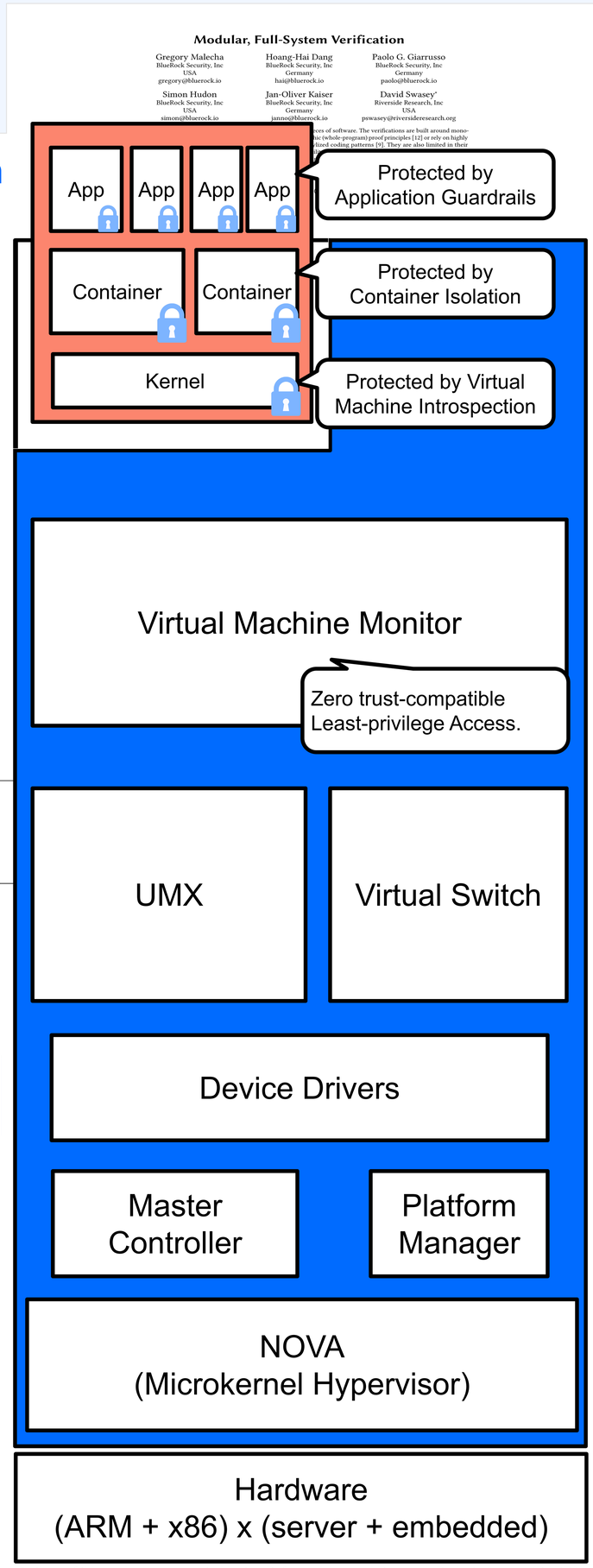
- **PL011 serial device** – Simple character I/O specification on top of the PL011 hardware. Proof supports interrupts (routed through NOVA) and manages the RX and TX queues ensuring that bytes are not lost.
- **ZynqMP DMA Device (simple mode)** – memcpy-like specification on top of a the ZynqMP hardware. Proof supports interrupts (routed through NOVA) and covers DMA accesses by the hardware.

Highly concurrent, capability-based micro-hypervisor.

Formal specification of NOVA & BRiCk-compatible specifications for C++ bindings are available.

Current verificationst include:

- Concurrent object spaces
- Semaphores, including timeouts and wait queues
- Kernel entry-points for capability creation & configuration. Verification roadmap includes: scheduler, page tables, IPC calls, etc



Portions of this work were supported by the Defense Advanced Research Projects Agency (DARPA)

The views, opinions, and/or findings expressed are those of the author(s) and should not be interpreted as representing the official views or policies of the Department of Defense or the U.S. Government.